

AtlasIT Prototype to Controlled MVP Roadmap

Executive summary

AtlasIT's codebase is accessible via the enabled GitHub ¹ connector, and within that scope the primary repository is a private monorepo: `JW-Flo/Project-AtlasIT` (default branch `main`). The repository already contains substantial "MVP-shaped" subsystems—especially the compliance/evidence layer (D1 + R2) with integration tests that validate deterministic hashing, idempotency, tenant isolation, evidence search pagination, retention purge endpoints, and health metric enrichment. ² ³ ⁴

However, AtlasIT is still in prototype mode because several critical "controlled MVP" hardening items are either incomplete or implemented as placeholders:

- The Joiner/Mover/Leaver workflow engine (`JMLEngine`) currently behaves as a minimal shim rather than a persistent workflow orchestrator; its behavior does not satisfy the expectations defined by the unit tests (Durable Object-like persistence + DLQ semantics). ⁵ ⁶
- The console has an explicit temporary auth bypass flag (`DISABLE_CONSOLE_AUTH="true"`) that must be removed or tightly constrained for any external pilot. ⁷
- The OpenAPI "verification" script is intentionally lightweight and does not ensure spec-to-implementation route parity; unsurprisingly, the public OpenAPI file and the compliance worker routes appear to diverge in path prefixes (e.g., `/api/policies/*` vs `/api/v1/policies/*`). ⁸ ⁹ ³
- Governance requirements requested for the next stage—OPA/Rego guardrails, Vault policy model, and a "single evidence schema" used consistently across workflow execution + policy evaluation—are partially designed in docs but not yet end-to-end enforced across all endpoints. ¹⁰ ¹¹

A controlled MVP is achievable in a short, phased sequence by (1) hardening CI/quality gates around the workflow engine and contract tests, (2) converting "synthetic workflow steps" into real adapter-driven actions with deterministic evidence emission, (3) adding production-grade access control and secrets governance (Vault + OIDC), and (4) operationalizing SLOs/alerts/runbooks using Cloudflare-native observability and SRE burn-rate alerting patterns. ¹²

Repository discovery and current codebase understanding

Enabled connectors (as requested): **GitHub**. Repository discovery was performed via the GitHub connector (installations show access under the authenticated user context). Within the connected installation scope, the AtlasIT repository located is `JW-Flo/Project-AtlasIT` (private).

Monorepo composition and build/test posture

The root `package.json` defines a workspaces-based monorepo that includes multiple Workers services (core, compliance, orchestrator, dispatch, docs), a SvelteKit console, and "tooling packages" such as an adapter generator and a research normalizer. ¹³

A CI workflow exists at `.github/workflows/ci.yml` and runs `validate:env` and `ci:run` (which builds shared code, runs Vitest unit tests, and generates an SBOM artifact directory). ¹⁴ ¹⁵

Key observations for “code health”:

- Type-checking is centralized via `tsc -p tsconfig.json --noEmit`. ¹³ ¹⁶
- Vitest is configured to include `**/*.test.ts` with modest coverage thresholds (lines/functions/statements 40%, branches 30%). ¹⁷
- There are evidence + compliance automation integration tests validating deterministic hashing and persistence behavior, which is a strong foundation for a controlled MVP. ² ³

Runtime architecture in the repository

AtlasIT is built primarily on Cloudflare ¹⁸ Workers and Cloudflare-native bindings:

- **Core Worker:** A top-level worker (`index.js` entry) implements `/health`, `/api/apps`, `/api/connectors`, etc., and is configured with KV, D1, R2, and dispatch namespace bindings in `wrangler.toml`. ⁵ ¹⁹
- **Workers for Platforms / Dispatch:** The repo is already configured to use dispatch namespaces (Workers for Platforms), aligning with Cloudflare’s “dynamic dispatch worker + dispatch namespace” model. ¹⁹ ²⁰
- **Compliance Worker:** A dedicated worker (`compliance-worker/src/index.ts`) provides endpoints for workflows execution, policies/templates, policy evaluation, evidence ingest/search/retrieve/verify, retention purge, and enriched health reporting. ²¹ ³
- **Console:** A SvelteKit console worker routes `www.atlasit.pro/console*` and related paths, binds D1 + KV, and proxies to the compliance worker. The current config intentionally disables auth (`DISABLE_CONSOLE_AUTH="true"`) and relies on Cloudflare Access headers when enabled. ⁷
- **Evidence & retention:** Evidence is written to R2 and indexed in D1; snapshots and retention are explicitly documented. ²² ¹⁰ ⁴

This design is consistent with Cloudflare’s binding model for R2 and D1 (bindings are declared in Wrangler config and accessed through `env.<BINDING>`). ²³

JML flows are defined but not yet “real”

The repository includes Joiner/Mover/Leaver fixtures and a workflow execution surface area that returns 4-step runs (joiner/mover/leaver) and persists step outputs (per integration tests), but the underlying “IAM engine” orchestration is not yet actually driving adapter operations end-to-end. ³ ²⁴ ²⁵

Separately, there is a `JMLEngine` class exported from the core worker, and a unit test suite expects this to behave like a persistent orchestrator (Durable Object semantics, retries, DLQ). The current `JMLEngine` implementation is only a minimal shim that returns a JSON response and does not implement the persistence behaviors the tests describe. ⁵ ⁶

This mismatch is the single biggest “prototype → MVP” blocker because controlled MVP requires (a) workflow state durability, (b) deterministic replayability, and (c) evidence emission at each step boundary.

Adapter generator + structured research engine exist and are MVP-relevant

AtlasIT already has a coherent “connector toolkit”:

- **Research Engine:** Converts Markdown “research notes” into a structured schema (and derived OpenAPI) that can be fed into generation. It produces artifacts for multiple researched services (example HR suite, Slack provisioning, Google Workspace admin). ²⁶ ²⁷
- **Adapter Generator:** Generates a Worker-compatible scaffold from either OpenAPI JSON or the research schema, sets feature flags, and updates an adapter registry. ²⁸
- **Adapter Simulator:** A CLI that currently validates `/health` and headers, producing JUnit output for CI. ²⁸
- **PR4/PR5 artifacts** demonstrate these toolchains were exercised and produced SBOM/JUnit logs for adapters and schemas. ²⁹ ²⁶

This is a strong differentiator: once workflow execution is real, AtlasIT can ship connectors faster with consistent contracts.

Controlled MVP definition and recommended product focus

North-star for the next stage

Based on the in-repo context summary, AtlasIT’s intended differentiation is “automation-first IT & security” for SMB/mid-market with identity lifecycle automation (JML), compliance automation (“Compliance Digital Twin”), and a unified automation console. ³⁰

A controlled MVP should therefore be defined as:

- **3–5 design partner tenants** (not self-serve), each with explicit tenant isolation, controlled feature flags, and a narrow connector set.
- A “thin but real” value loop: **JML automation → evidence emission → compliance snapshot → human-visible console + export.**
- A security posture that is credible for pilot customers: authenticated console access, strong secrets model, and auditable policy decisions.

Recommended vertical and MVP package

Given the existing compliance/evidence subsystems and rule evaluation scaffolding, the most natural near-term vertical is:

Identity & access governance (IGA-lite) + continuous evidence for SOC2 access controls (starting with identity lifecycle events and access request workflows).

This positioning competes with continuous compliance platforms like Drata ³¹ and Vanta ³² on evidence automation, and with identity incumbents like Okta ³³ on lifecycle signals—but the wedge is “workflow automation + evidence packs” rather than being a full IdP. ³⁰

Controlled MVP package (concrete):

- **Tenant onboarding + RBAC:** tenant-scoped API tokens (already modeled in integration tests) + console access restricted via Cloudflare Access email allowlist. [3](#) [7](#)
- **JML flows:** joiner/mover/leaver execution with persistence, retries, and deterministic step logs.
- **Evidence emission:** each workflow step emits evidence into the evidence pipeline (R2 + D1 index) and links relevant evidence to controls.
- **Two “real” connectors** to start:
 - Identity provider: Okta (upgrade from offline adapter to real integration)
 - Communication/approvals: Slack (approval + notification surface)
- **Console views:** compliance snapshot, evidence search + evidence detail, workflows execution list + drilldown.

This is small enough to deliver but large enough to prove the flywheel.

Gap analysis and required artifacts for controlled MVP

The table below compares observed repo state vs. the artifacts/guarantees required for a next-stage controlled MVP.

Area	Current repo evidence	Controlled MVP requirement
Workflow engine durability	<code>JMLEngine</code> is currently a lightweight shim; tests require Durable Object-like persistence and DLQ behavior. 5 6	Implement a persistent orchestrator (Durable Object recommended) with explicit run state schema, retry policy, DLQ entries, and replay/export APIs.
Compliance & evidence backbone	Evidence ingest + canonical hashing + duplicate detection + search pagination + health metrics are validated via integration tests; retention is designed and partially implemented. 2 4	Make evidence emission mandatory for workflows and policy evaluation, enforce immutability (<code>If-None-Match</code> style), and ensure tenantId is always authenticated and non-optional.
API contracts	OpenAPI exists; verify script checks only a subset of paths/schemas. 8 9	Contract tests that fail CI when route handlers diverge from OpenAPI (generated route map or snapshot tests).
Console security	Console config has a deliberate auth bypass and trusts Access headers when enabled. 7	Remove bypass; require Cloudflare Access or equivalent OIDC session; add audit logging for admin actions.
Connectors and adapters	Adapter generator + research engine + simulator exist; simulator mainly checks <code>/health</code> . 28	Define a versioned adapter contract (auth, idempotency, error schema, rate limits). Expand simulator to validate core lifecycle operations and evidence emission hooks.

Area	Current repo evidence	Controlled MVP requirement
Governance via policy	Repository has a “policy & evidence” design doc; enforcement is mostly application-level at present. ¹⁰	Externalize decision logic into OPA/Rego bundles (or OPA-wasm) and record decision logs as evidence. ³⁴
Secrets management	Secrets checklist exists; oriented around GitHub secrets + Wrangler secrets, not Vault. ³⁵	Adopt Vault policies + OIDC/JWT auth for CI/CD; reduce long-lived secrets in GitHub by retrieving short-lived secrets at deploy time. ³⁶
Observability + SLOs	Workers observability is enabled in configs; tracing/logging patterns exist, but SLO/alert/runbook system is not fully productized. ¹⁹ ³⁷	Define SLOs and adopt burn-rate based alerting (multiwindow, multi-burn-rate); publish runbooks and link them from alerts. ³⁸

Phased roadmap to a controlled MVP

The roadmap below is designed to take AtlasIT from the current prototype to a controlled MVP with explicit exit criteria, owners, and artifacts. The time estimates assume a small team (1–3 engineers) and prioritize de-risking (workflow durability + auth + evidence correctness) before adding breadth.

Architecture target for controlled MVP

```

flowchart LR
    U[Design Partner Admin] --> C[Console Worker / SvelteKit]
    C -->|API calls| CORE[Core API Worker]
    C -->|Compliance views| COMP[Compliance Worker]

    CORE -->|dynamic dispatch| DISP[Dispatch Worker]
    DISP --> NS[Dispatch Namespace<br/>Customer/Adapter Workers]

    CORE --> DO[JML Orchestrator<br/>Durable Object]
    DO --> COMP

    COMP --> D1[(D1: compliance + evidence index)]
    COMP --> R2[(R2: immutable evidence + snapshots)]
    CORE --> KV[(KV: tokens/flags/sessions)]

    subgraph Governance
        POL[OPA policies<br/>bundle/wasm]
        COMP --> POL
        POL -->|decision logs| R2
    end
end

```

Cloudflare Workers for Platforms dispatch namespaces are an appropriate mechanism for isolating generated adapter workers and invoking them dynamically (`env.<NAMESPACE>.get("worker-name")`).

Evidence storage should continue to follow Cloudflare's R2 binding model and D1 binding model for worker-side access. ³⁹

Milestone timeline

```

gantt
  title AtlasIT Controlled MVP Timeline (starting 2026-03-16)
  dateFormat YYYY-MM-DD

  section Foundation hardening
  Fix JMLEngine persistence + test parity           :a1, 2026-03-16, 10d
  Replace console auth bypass, enforce Access/OIDC :a2, 2026-03-16, 7d
  Stronger contract tests (OpenAPI parity)         :a3, 2026-03-20, 10d

  section Workflow-to-evidence loop
  Step-level evidence emission + linking           :b1, 2026-03-26, 10d
  Real adapter invocation from workflows           :b2, 2026-03-30, 12d

  section Governance and security
  OPA policy bundle/wasm integration + decision logs:b3, 2026-04-07, 12d
  Vault + GitHub OIDC for deploy secrets           :b4, 2026-04-10, 10d

  section Operational readiness
  SLOs + burn-rate alerts + runbooks               :c1, 2026-04-18, 10d
  Pilot cut + reliability gate                     :c2, 2026-04-26, 7d

```

The burn-rate, multiwindow approach is explicitly recommended in the SRE Workbook “Alerting on SLOs” chapter and is appropriate for AtlasIT’s API availability/latency objectives. ⁴⁰

Prioritized task list with owners and acceptance criteria

The list below is ordered by “pilot risk reduction” (highest first). Effort is expressed as **S/M/L** (roughly 1–2d / 3–5d / 6–10d for one engineer) or story points.

Implement real JML orchestrator semantics (Persistent + replayable)

Owner: Platform Engineering

Effort: L (13–20 SP)

Acceptance criteria: - `tests/jml-engine.test.ts` passes without skips; run state is persisted across calls and DLQ entries are created when failing steps exceed retry limits. ⁶

- Run state schema is versioned and serialized deterministically (so replays reproduce identical hashes for identical inputs).

- Durable scheduling is supported (Durable Object alarms preferred) to avoid relying on Worker queues/crons limits during early pilots. (Durable Objects are designed for stateful coordination workloads.) ⁴¹

Make workflow steps “real” by invoking adapters, not synthetic results

Owner: Platform Engineering + Integrations

Effort: L (13–20 SP)

Acceptance criteria: - Workflow step actions call adapter workers through the dispatch namespace model and record outputs per step (status, attempts, duration). ⁴²
- Adapter call failures trigger per-step retry policy and eventual DLQ behavior.

Evidence emission becomes a first-class contract for workflow steps

Owner: Platform Engineering

Effort: M (8–13 SP)

Acceptance criteria: - Every workflow step emits an evidence envelope (or evidence artifact reference) using canonical hashing and is written to R2 + D1 index. ² ¹⁰
- Evidence writes are idempotent and immutable, and step logs contain the evidence hash.

Remove console auth bypass and establish controlled pilot access

Owner: Frontend + Security

Effort: M (5–8 SP)

Acceptance criteria: - `DISABLE_CONSOLE_AUTH` is removed or forced false in production routes; console login relies on Cloudflare Access headers or an equivalent OIDC mechanism. ⁷
- Admin actions (workflow execute, retention purge, policy changes) are auditable via structured logs (and optionally stored as evidence).

Strengthen “contract-first” enforcement of OpenAPI vs handlers

Owner: DevEx

Effort: M (5–8 SP)

Acceptance criteria: - Replace the lightweight OpenAPI check with an API surface test that validates all documented paths exist and respond with compatible shapes for at least one golden request per endpoint group. ⁹
- CI fails on breaking changes unless versioning + changelog rules are followed (align with the phased roadmap doc). ⁴³

OPA/Rego governance for high-stakes decisions

Owner: Security + Platform Engineering

Effort: M/L (8–13 SP)

Acceptance criteria: - Policies are shipped as signed bundles or a reproducible artifact and loaded into an evaluation runtime; updates do not require redeploy of the business logic worker. (OPA bundles are designed for dynamic, on-the-fly policy updates.) ⁴⁴
- Decision logs (input/result/bundle revision) are emitted for auditing, either to R2 or as evidence envelopes (OPA supports structured decision logging). ⁴⁵

Vault + OIDC-based secret retrieval for CI/CD

Owner: Security + DevEx

Effort: M (5–8 SP)

Acceptance criteria: - GitHub Actions authenticates to Vault using GitHub's OIDC provider (`https://token.actions.githubusercontent.com`) via Vault's JWT auth method. ³⁶
- Wrangler deploy tokens and third-party API keys are retrieved at job runtime rather than stored as long-lived GitHub secrets, except for minimal bootstrap secrets.

SLOs, alerts, and runbooks

Owner: SRE/Platform

Effort: M (5–8 SP)

Acceptance criteria: - Define initial SLOs (availability + p95 latency) per critical endpoint group (workflow execute, evidence ingest, compliance snapshot).

- Implement burn-rate alerting with multiwindow strategy per SRE Workbook guidance. ⁴⁰
- Publish runbooks with “diagnose → mitigate → recover” steps; link runbooks from alerts and console.

Cloudflare supports Workers Logs (structured JSON recommended) and Analytics Engine datasets for metrics-like events, which can back SLO calculations and burn-rate alert logic. ⁴⁶

Implementation templates and “starter” snippets

This section provides concrete artifacts you can add as PRs to accelerate MVP hardening.

PR template for common issues (TypeScript failures, tests, evidence)

```
## Summary
- [ ] What problem does this PR solve?
- [ ] What user-facing/ops-facing behavior changes?

## Type safety
- [ ] `npm run typecheck` passes (tsc --noEmit)
- [ ] No new `any` leakage in public interfaces
- [ ] Added/updated types for env bindings

## Tests
- [ ] `npm run test:unit` passes
- [ ] Added/updated unit tests for new logic
- [ ] Added/updated integration tests if touching evidence/workflows

## Evidence & auditability (if applicable)
- [ ] New workflow steps emit evidence envelopes
- [ ] Evidence hash is stable under key reorder (canonical JSON)
- [ ] TenantId is authenticated and cannot be spoofed

## Ops
- [ ] Updated docs/runbooks if behavior changed
- [ ] Added/updated alerts/SLO calculations if SLI changed

## Rollback plan
- [ ] How do we revert safely?
```

This aligns with the repo’s existing emphasis on canonical hashing and deterministic evidence. ² ⁴⁷

Common fix pattern for `tsc --noEmit` drift in a monorepo

Recommended changes (high leverage, low risk):

- Add per-package `tsconfig.build.json` and run `tsc -b` with project references for packages that ship runtime artifacts (idp, idp-okta, adapter-gen, research-engine).
- Keep tests out of build configs but include them in a separate `tsconfig.test.json` so Vitest compilation errors can be caught by TypeScript too (root `tsconfig.json` currently excludes `*.test.ts`). ¹⁶

Sample Evidence JSON Schema for “unified evidence envelope”

The repository already documents an “evidence envelope” shape and canonical hashing expectations. ¹⁰

Below is a production-usable JSON Schema sketch you can put under `schemas/evidence-envelope.schema.json` and enforce at ingest boundaries:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://atlasit.pro/schemas/evidence-envelope.schema.json",
  "title": "AtlasIT Evidence Envelope",
  "type": "object",
  "required": ["tenantId", "policyPack", "subject", "result", "createdAt"],
  "properties": {
    "id": { "type": "string", "description": "Optional UUID for envelope tracing" },
    "tenantId": { "type": "string" },
    "policyPack": {
      "type": "object",
      "required": ["name", "version"],
      "properties": {
        "name": { "type": "string" },
        "version": { "type": "string" },
        "checksum": { "type": "string", "description": "sha256:<hex> of the pack source" }
      },
      "additionalProperties": false
    },
    "subject": {
      "type": "object",
      "properties": {
        "userId": { "type": "string" },
        "deviceId": { "type": "string" },
        "attributes": { "type": "object", "additionalProperties": true }
      },
      "additionalProperties": true
    },
    "result": {
```

```

    "type": "object",
    "properties": {
      "allow": { "type": "boolean" },
      "requirements": { "type": "array", "items": { "type": "string" } },
      "violations": { "type": "array", "items": { "type": "string" } }
    },
    "additionalProperties": true
  },
  "artifacts": {
    "type": "array",
    "items": {
      "type": "object",
      "required": ["kind", "hash"],
      "properties": {
        "kind": { "type": "string" },
        "hash": { "type": "string", "pattern": "^sha256:[a-f0-9]{64}$" },
        "path": { "type": "string" }
      },
      "additionalProperties": false
    },
    "hash": {
      "type": "string",
      "pattern": "^[a-f0-9]{64}$",
      "description": "sha256 of canonicalized envelope (without storage fields)"
    },
    "createdAt": { "type": "string", "format": "date-time" }
  },
  "additionalProperties": true
}

```

Sample OPA/Rego snippet for workflow authorization

OPA bundles allow policy and data updates without redeploying the calling service, which is attractive for controlled pilots where governance evolves quickly. ⁴⁸

The Rego language supports an explicit “default deny” allow rule style. ⁴⁹

```

package atlasit.authz

default allow := false

# Allow workflow execution only to principals with role "automation:execute"
allow if {
  input.action == "workflow.execute"
  some r
  input.subject.roles[r] == "automation:execute"
}

```

```

    input.tenantId == input.subject.tenantId
  }

# Deny retention purge unless "admin:retention" present
deny[msg] if {
  input.action == "retention.purge"
  not input.subject.roles[_] == "admin:retention"
  msg := "missing required role admin:retention"
}

```

Operational recommendation: enable decision logs and treat each decision log entry as evidence (store in R2 or index via D1). OPA decision logs include input, result, and policy/bundle metadata, which supports auditing and replay. ⁴⁵

Sample Vault policy + GitHub OIDC configuration sketch

Vault's JWT/OIDC auth method supports authenticating using GitHub Actions OIDC tokens, and GitHub's own guidance shows configuring Vault with issuer/discovery URL `https://token.actions.githubusercontent.com`. ⁵⁰

Vault policy (HCL) example (least privilege for deploy jobs):

```

# Read-only access to production deploy secrets for AtlasIT Workers
path "kv/data/atlasit/prod/cloudflare" {
  capabilities = ["read"]
}

path "kv/data/atlasit/prod/third-party" {
  capabilities = ["read"]
}

```

Vault JWT auth role constraints (conceptual): - Bind `aud` to your GitHub org - Bind repository and branch claims so only `JW-Flo/Project-AtlasIT` on `main` can deploy - Issue short-lived Vault tokens (5–15 minutes) for deploy steps

This reduces reliance on long-lived GitHub secrets while keeping deploy pipelines auditable and revocable.

Observability baseline: Cloudflare-native instrumentation

Cloudflare Workers Logs can capture invocation logs and structured JSON logs; structured logging is explicitly recommended for queryability, and sampling is configurable via `head_sampling_rate`. ⁵¹ Cloudflare Workers Analytics Engine can store “datapoints” and be queried via SQL/GraphQL for dashboards and SLO calculations. ⁵²

Practical controlled-MVP guidance: - Emit one Analytics Engine datapoint per workflow execution (duration, status, tenantId hash, workflow type). - Emit one datapoint per evidence ingest (stored vs duplicate, payload size, pack). - Define SLOs and burn-rate alerts using the SRE Workbook's Chapter 5 patterns (multiwindow/multi-burn-rate). 40

8 openapi.yaml

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/docs/api/openapi.yaml>

2 evidence.test.ts

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/tests/integration/evidence.test.ts>

3 compliance-automation.test.ts

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/tests/integration/compliance-automation.test.ts>

4 DATA_RETENTION_MATRIX.md

https://github.com/JW-Flo/Project-AtlasIT/blob/main/docs/DATA_RETENTION_MATRIX.md

5 index.js

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/index.js>

6 cli.ts

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/packages/adapters/src/cli.ts>

7 31 wrangler.toml

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/console-app/wrangler.toml>

9 openapi-verify.mjs

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/scripts/openapi-verify.mjs>

10 POLICY_AND_EVIDENCE.md

https://github.com/JW-Flo/Project-AtlasIT/blob/main/docs/POLICY_AND_EVIDENCE.md

11 33 34 44 48 <https://www.openpolicyagent.org/docs/management-bundles>

<https://www.openpolicyagent.org/docs/management-bundles>

12 37 46 51 <https://developers.cloudflare.com/workers/observability/logs/workers-logs/>

<https://developers.cloudflare.com/workers/observability/logs/workers-logs/>

13 package.json

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/package.json>

14 ci.yml

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/.github/workflows/ci.yml>

15 package.json

<https://github.com/JW-Flo/Project-AtlasIT/blob/2c12e4b8e857c18b6d9aaf8d179643f974531cdc/package.json>

16 tsconfig.json

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/tsconfig.json>

17 vitest.config.ts

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/vitest.config.ts>

27 package.json

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/templates/worker/package.json>

19 **wrangler.toml**

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/wrangler.toml>

20 42 **<https://developers.cloudflare.com/cloudflare-for-platforms/workers-for-platforms/how-workers-for-platforms-works/>**

<https://developers.cloudflare.com/cloudflare-for-platforms/workers-for-platforms/how-workers-for-platforms-works/>

21 **registry.json**

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/adapters/registry.json>

22 **0001_init.sql**

https://github.com/JW-Flo/Project-AtlasIT/blob/main/compliance-worker/migrations/0001_init.sql

23 39 **<https://developers.cloudflare.com/r2/api/workers/workers-api-usage/>**

<https://developers.cloudflare.com/r2/api/workers/workers-api-usage/>

24 **mover.json**

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/fixtures/jml/mover.json>

25 **leaver.json**

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/fixtures/jml/leaver.json>

26 **RUN.json**

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/artifacts/research/RUN.json>

28 **CONNECTORS.md**

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/docs/CONNECTORS.md>

29 32 **RUN.json**

https://github.com/JW-Flo/Project-AtlasIT/blob/main/artifacts/pr4_adapter/RUN.json

30 **# AtlasIT – Project Context Summary.md**

[https://github.com/JW-Flo/Project-AtlasIT/blob/main/docs/# AtlasIT – Project Context Summary.md](https://github.com/JW-Flo/Project-AtlasIT/blob/main/docs/#%20AtlasIT%20%E2%80%93%20Project%20Context%20Summary.md)

35 **DEPLOYMENT_SECRETS_CHECKLIST.md**

https://github.com/JW-Flo/Project-AtlasIT/blob/main/ops/DEPLOYMENT_SECRETS_CHECKLIST.md

36 50 **<https://docs.github.com/en/actions/how-tos/secure-your-work/security-harden-deployments/oidc-in-hashicorp-vault>**

<https://docs.github.com/en/actions/how-tos/secure-your-work/security-harden-deployments/oidc-in-hashicorp-vault>

38 40 **<https://sre.google/workbook/alerting-on-slos/>**

<https://sre.google/workbook/alerting-on-slos/>

41 **<https://workers.cloudflare.com/product/durable-objects>**

<https://workers.cloudflare.com/product/durable-objects>

43 **roadmap.md**

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/docs/roadmap.md>

45 **<https://www.openpolicyagent.org/docs/management-decision-logs>**

<https://www.openpolicyagent.org/docs/management-decision-logs>

47 **canonical-json.ts**

<https://github.com/JW-Flo/Project-AtlasIT/blob/main/src/lib/canonical-json.ts>

⁴⁹ <https://www.openpolicyagent.org/docs/policy-language>

<https://www.openpolicyagent.org/docs/policy-language>

⁵² <https://developers.cloudflare.com/analytics/analytics-engine/get-started/>

<https://developers.cloudflare.com/analytics/analytics-engine/get-started/>